UNIVERSITÄT LEIPZIG

07.05.2026
Jun.-Prof. Dr. Leonie Weißweiler

Natural Language Processing

Exercise Sheet 5: Transformers – Part 1

1 Perceptron

Consider the simple perceptron P in Figure 1, which takes two variable x_1 and x_1 as its input and computes a binary prediction $y_{pred} \in \{-1, 1\}$, using the weights w_i associated with the inputs and the bias term b, by passing the sum of the weighted products through the activation function $g(z)$.

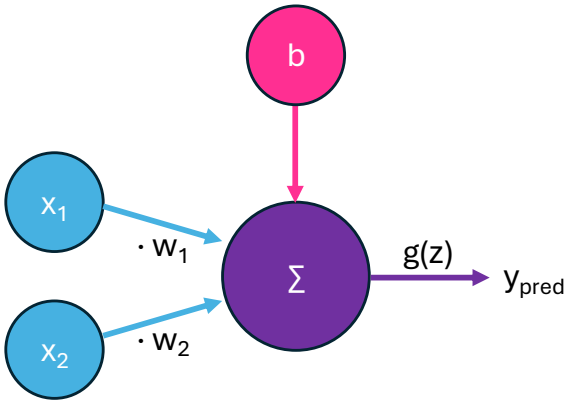

Figure 1: Perceptron with two inputs x_1 and x_2, and a bias term b.

Assuming these initial values and function definitions,

$$b = 1 \qquad \begin{matrix} w_1 = 2 \\ w_2 = 3 \end{matrix} \qquad z = \sum_i w_i x_i + b \qquad g(z) = \begin{cases} 1 & \text{if z} \geq 0 \\ -1 & \text{if z} < 0 \end{cases}$$

we will use the perceptron to predict an answer for the question

- "Will I go have ice cream after the tutorial?"

with two possible output scenarios, $y_{pred} = 1$ if you decide to buy ice cream and $y_{pred} = -1$ if you decide against buying ice cream, based on two input variables:

- x_1 = current temperature (C)
- x_2 = money (€) left in this week's budget

and we have two training samples $s = (x_1, x_2)$, along with the desired output y_{gold}:

- $s_1 = (25, 50) \rightarrow 1$
- $s_2 = (20, 10) \rightarrow -1$

1.1 Forward pass

Using the above definitions, calculate z and apply $g(z) = y_{pred}$ for the two training samples s_1 and s_2 by hand.

Evaluate your results with regard to y_{gold}.

1.2 Backward pass

For both samples, check whether a weight update should occur by computing the error/loss using the Loss function, $L = \max(0, -y_{gold} \cdot y_{pred})$.

If the weights should be updated, calculate the new weights $w_{i,t+1}$ using the update function and a learning rate of $\eta = 0.5$.

Update function: $w_{i,t+1} = w_{i,t} + \eta \times y_{gold} \times x_i$

hint: Treat b as a weight without a unique x_i variable.

Beginning with these new weights $w_{i,t+1}$, continue to calculate forward passes, examine loss values, and update the weights until the perceptron converges.

hint: Convergence will occur within less than 5 updates.

2 Simple FFNN

In this section, we will consider the opinion of another classmate. If you and your classmate have the same opinion about whether or not to buy an ice cream, you can go to an ice cream shop together or go back home together after the tutorial. Thus, the input to the model is:

$$(x_1, x_2) = (\text{your decision}, \text{your friend's decision})$$

The decisions are defined by a binary class: 0 if the person is not going to buy an ice cream, 1

if the person is going to buy an ice cream.

For example, if you don't want to buy ice cream but your friend wants to buy one, it will be expressed as $(x_1, x_2) = (0, 1)$.

For the output, we want to obtain a binary value y_{pred} about whether you two have the same opinion or not. Thus, the output y_{pred} will be 1 if you disagree with each other, and 0 if you agree with each other.

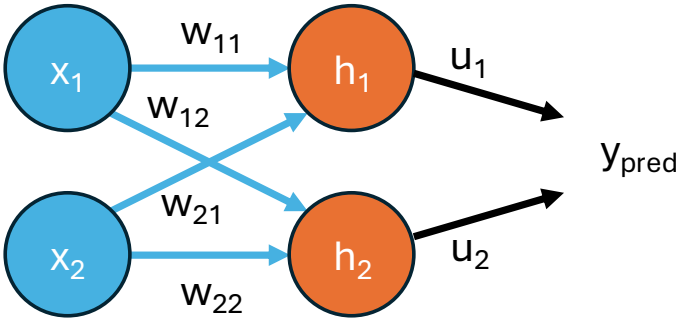

Figure 2: Two by two feed-forward network without a bias term. ReLU functions are not described in this figure for simplicity.

Figure 2 shows the 2×2 feed forward network. The ReLU functions are not described for simplicity. We represent the input and hidden vectors as column vectors:

$$X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad H = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix}$$

The weights from the hidden layer to the output are

$$U = \begin{bmatrix} u_1 & u_2 \end{bmatrix}$$

and the weight matrix from the input layer to the hidden layer is

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}$$

Note that we do not consider bias terms in this section. The network can be described by the following equations:

$$H = \mathrm{ReLU}(WX)$$

$$y_{\mathrm{pred}} = \mathrm{ReLU}(UH)$$

ReLU function gives $y = x$ if $x \geq 0$, otherwise 0.

As initial values for weights, we have the following.

$$W = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & 1 \end{bmatrix}$$

2.1 Forward pass

Calculate the forward pass for all possible (x_1, x_2). Discuss whether our current model works correctly.

hint: Perform matrix multiplication, or alternatively from Figure 2:

$$h_1 = \text{ReLU}(w_{11}x_1 + w_{12}x_2) \quad (1)$$

$$h_2 = \text{ReLU}(w_{21}x_1 + w_{22}x_2) \quad (2)$$

$$y_{\text{pred}} = \text{ReLU}(u_1h_1 + u_2h_2) \quad (3)$$

2.2 Backward pass

We want to update the model weights by one backward pass so that the model would correctly assign the output y_{pred} to any (x_1, x_2). The loss function we use is the Mean Squared Error (MSE) loss, denoted as

$$L = \frac{1}{2}(y_{\text{gold}} - y_{\text{pred}})^2 \quad (4)$$

Use the learning rate $\eta = 1$, and only update w_{12} following the update rule: $w_{12} \leftarrow w_{12} - \eta \frac{\partial L}{\partial w_{12}}$.

The derivative of the ReLU function is as follows:

$$\text{ReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

hint: if we use (x_1, x_2) for which the model already assigns the correct output value, the MSE loss will be 0, for which there is no need to update any weight. Thus, use the pair (x_1, x_2) which did not lead to the expected output value in subsection 2.1. In order to update, we need to know the value of $\frac{\partial L}{\partial w_{ij}}$. We show the necessary calculations below.

Using the chain rule,

$$\frac{\partial L}{\partial w_{12}} = \frac{\partial L}{\partial y_{\text{pred}}} \cdot \frac{\partial y_{\text{pred}}}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_{12}} \quad (5)$$

by taking the derivatives of (1), (3), (4),

$$\frac{\partial L}{\partial y_{\text{pred}}} = y_{\text{pred}} - y_{\text{gold}} \quad (6)$$

$$\frac{\partial y_{\text{pred}}}{\partial h_1} = \text{ReLU}'(u_1 h_1 + u_2 h_2)\, u_1 \quad (7)$$

$$\frac{\partial h_1}{\partial w_{12}} = \text{ReLU}'(w_{11} x_1 + w_{12} x_2)\, x_2 \quad (8)$$

Now we have all the pieces to update the weight w_{12}. Use (6), (7), (8) to get the value necessary for (5) and update the weight.

Exercise 3

Have a go at this gradient descent treasure hunt fishing game (1 player, "ohne"/without opponent or choose one of the algorithms for your opponent), and win using your knowledge of gradient descents and function optimisation: `https://ki-campus.org/simulations/gradient-descent`